SLIME SLAYER

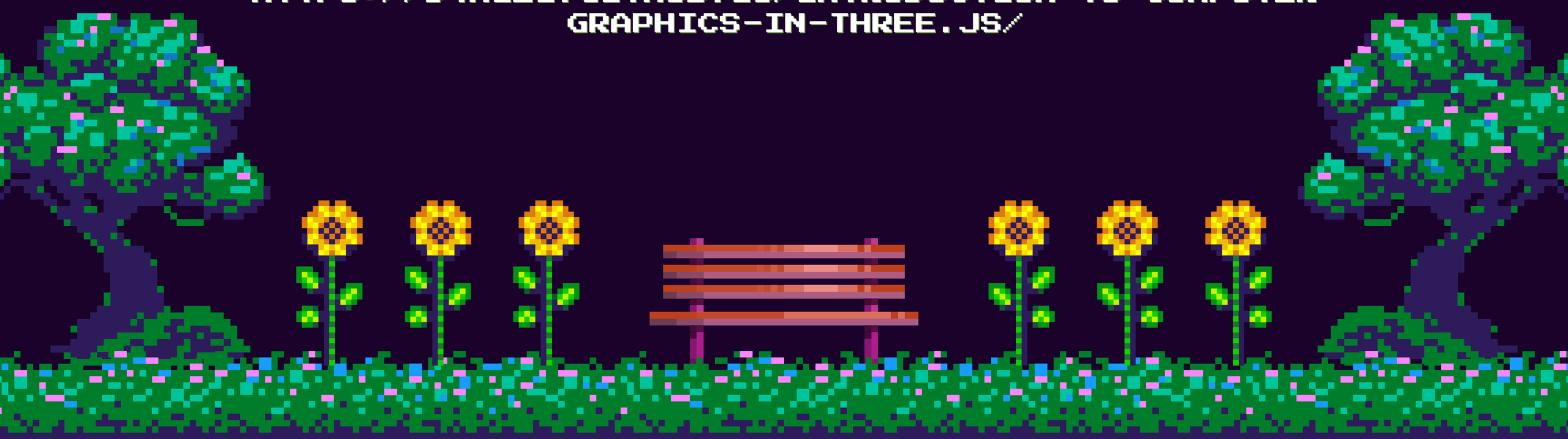
DANIEL MARTINS - 115868

INTRODUCTION TO COMPUTER GRAPHICS - 2025/2026 - FINAL PROJECT

LINKS

https://github.com/d4n1ei/introduction-to-computer-graphics-in-three.js

https://d4n1ei.github.io/introduction-to-computer-graphics-in-three.js/

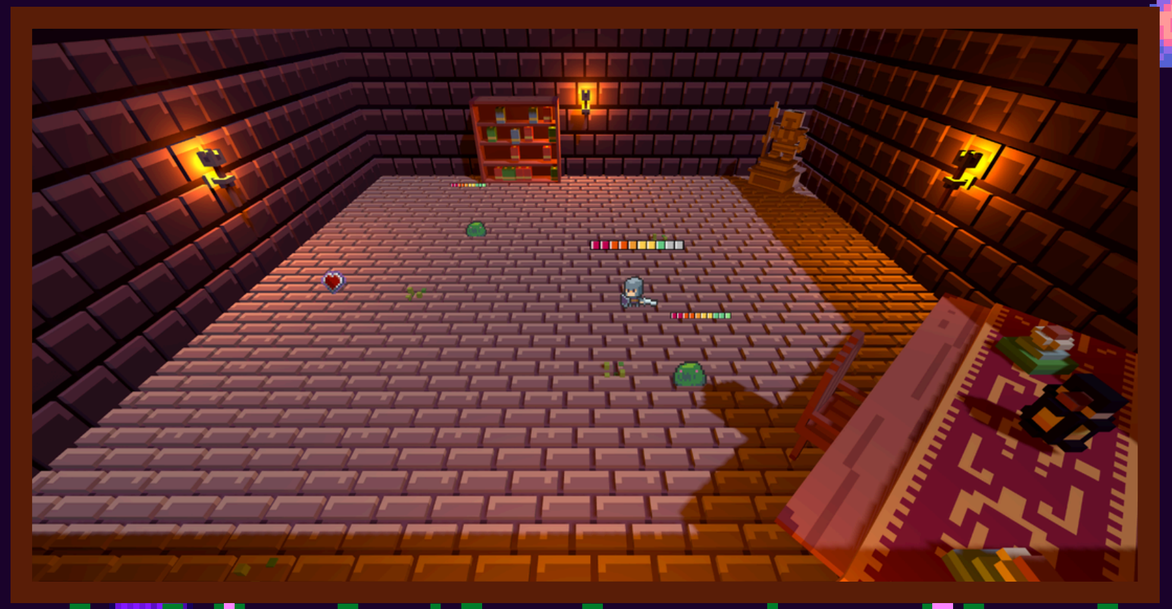


ABOUT THE GAME

Simple Three.js TypeScript game where you play a soldier, defeat 5 spawning slimes, and manage health by avoiding damage and collecting periodic health packs. You win by killing 5 slimes and lose if your health drops to 0.

Used for its OO Design features and static typing

Used for endering interactive 3D graphics in the browser easily using WebGL.



HOW TO PLAY THIS GAME?

W
ASD

Movement is controlled using the WASD keys, and the spacebar allows the player to jump, although this feature has no gameplay effect since verticality was not implemented.

F

You may perform an attack using the F key.

Class responsible for handling player input

```typescript
export class InputSystem implements IEventDrivenSystem{
    private keys: Record<string, boolean> = {};

    constructor() {
        window.addEventListener("keydown", (e) => {
            this.keys[e.key.toLowerCase()] = true;
        });

        window.addEventListener("keyup", (e) => {
            this.keys[e.key.toLowerCase()] = false;
        });
    }

    isKeyDown(key: string): boolean {
        return !!this.keys[key];
    }
}
```

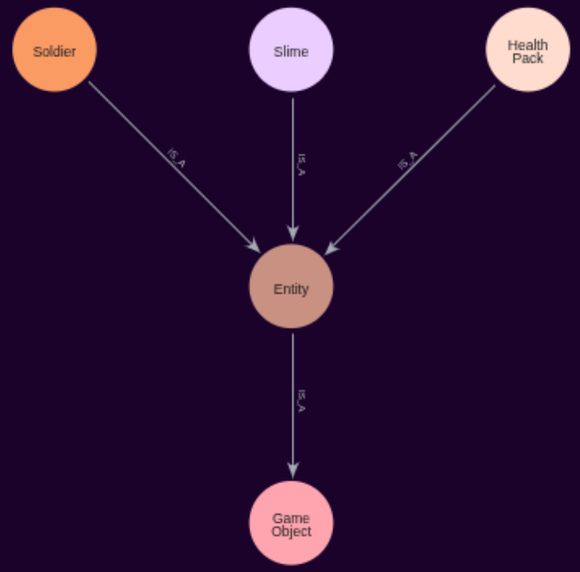
SCENE SYSTEM

Class that manages the 3D scene lifecycle in a Three.js game engine
Implements IUpdatableSystem → participates in the game loop

Central hub for:
- Rendering
- Managing Cameras
- Adding Lights
- Controlling Game objects

```ts
export class SceneSystem implements IUpdatableSystem {
  public scene: THREE.Scene;
  public cameras: THREE.Camera[] = [];
  public lights: THREE.Light[] = [];
  public gameObjects: GameObject[] = [];
  public activeCamera: THREE.Camera | null = null;

  private renderer: THREE.WebGLRenderer;
```

SO WHAT IS A GAME OBJECT?



The GameObject class is simply a container that stores an Object3D and a tag used as its identifier. It can also include a CollisionComponent, which will be explained later in the presentation.

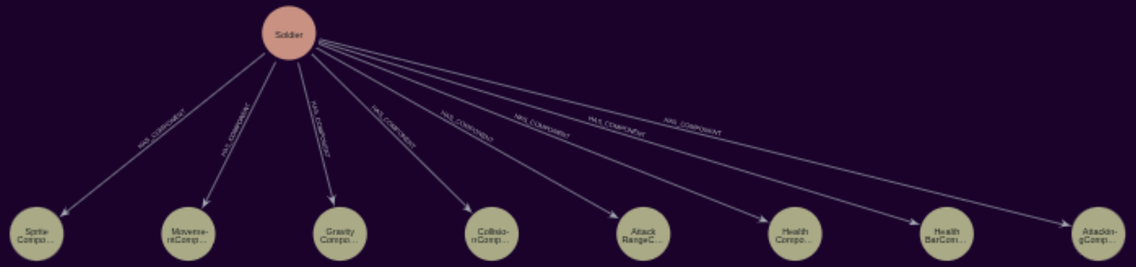
```
export class GameObject {
    object3D: THREE.Object3D;
    collisionComponent?: CollisionComponent;
    tag: string;
```

The classes are organized in an hierarchical structure. An Entity is a GameObject and Slime, Soldier and HealthPack are all Entities.

ENTITIES

Entities are an abstract concept that hold a series of Components. These components can be used to “decorate” sub-classes with gameplay mechanics.

```ts
export abstract class Entity extends GameObject {
  components: Map<string, IComponent> = new Map();
```



COMPONENTS

A component is a modular piece of data and behavior that adds a specific capability to a game object or entity.
It does not define a full object by itself - instead, it defines one aspect of how something behaves in the game.

```
export class CollisionComponent implements IUpdatableComponent {
  object3D: THREE.Object3D;

  collisionBox: THREE.Box3;
  collisionBoxHelper?: THREE.Box3Helper;

  private localBox?: THREE.Box3;
  private useMeshBounds: boolean;

  isOnGround: boolean;
```

A COMPONENT ISOLATES A SINGLE CONCERN (FEATURE) FROM THE REST OF THE SYSTEM, ALLOWING THE SOFTWARE TO SCALE BETTER, INSTEAD OF RELYING PURELY ON INHERITANCE, WE RELY ON COMPOSITION AND DEPENDENCY INJECTION TO STRUCTURE GAME ACTORS.

THEY'RE INJECTED INTO THE ENTITY IN THEIR CORRESPONDING FACTORIES.

- ATTACKINGCOMPONENT
- ATTACKRANGECOMPONENT
- COLLISIONCOMPONENT
- GRAVITYCOMPONENT
- HEALTHBARCOMPONENT
- HEALTHCOMPONENT
- MOVEMENTCOMPONENT
- SPRITECOMPONENT

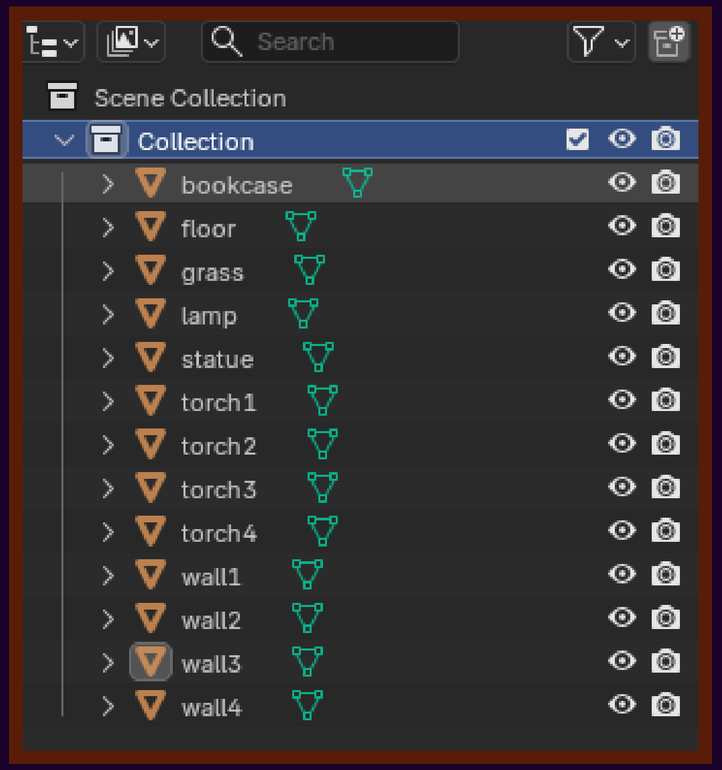
SYSTEMS

```ts
enum SlimeState {
    Wander,
    Breathing,
    Seek,
}

export class SlimeMovementSystem implements IUpdatableSystem {

    movement: MovementComponent;

    private state: SlimeState = SlimeState.Wander;

    private currentDirection: THREE.Vector3;
    private timer: number = 0;

    private walkDuration: number = 1.5;
    private breatheDuration: number = 2;
    private speed: number = 0.4;
    private player : Soldier;
```

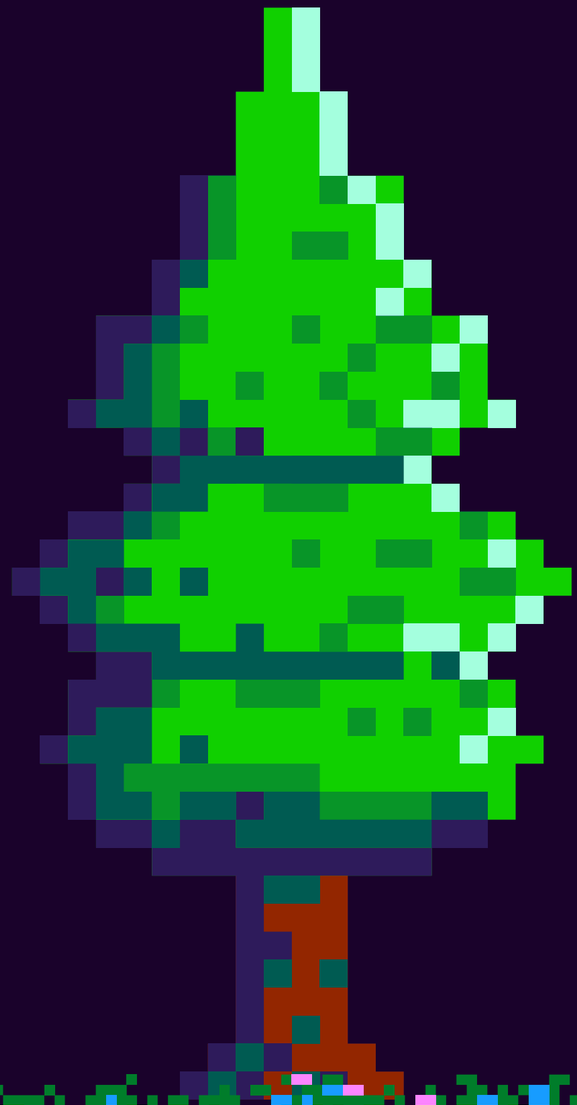
A system is a processor that runs game rules on components every frame.
It:

- Reads components
- Updates them
- Applies game logic
- Coordinates behavior across entities

GAME MAP



The assets were sourced from the web, while the overall design was developed independently. The objects were already textured, and I imported the collection and built upon it. The map is divided into multiple individual objects, which proved useful, as this separation facilitated traversal and manipulation of the GLB model in Three.js

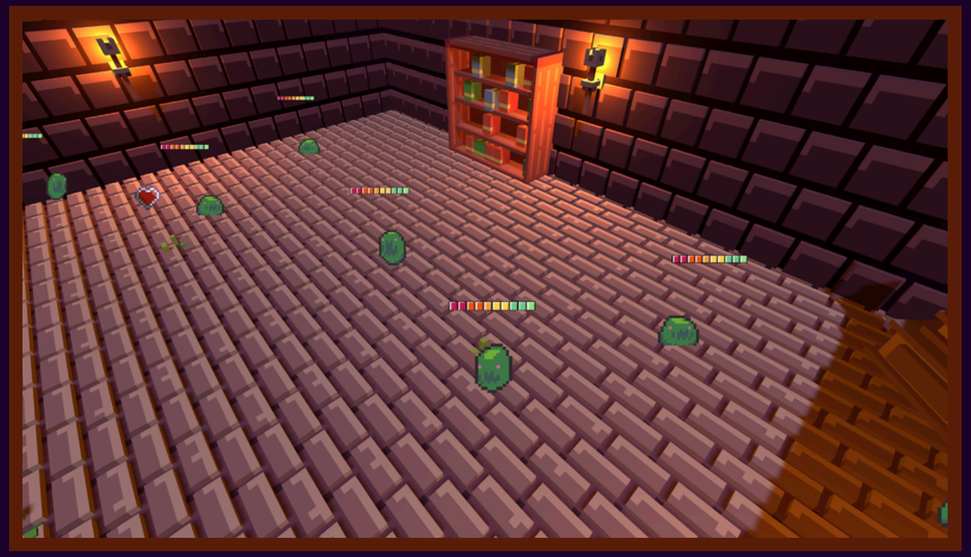


ILLUMINATION

The four torches present in the scene also emit a faint orange glow, which affects objects around them.

All illumination in the scene is handled by the SceneSystem. The lights are configured to cast shadows, and scene objects are able to receive shadows as well. Additionally, the scene includes a background that defaults to a black skybox and incorporates a subtle fog effect.

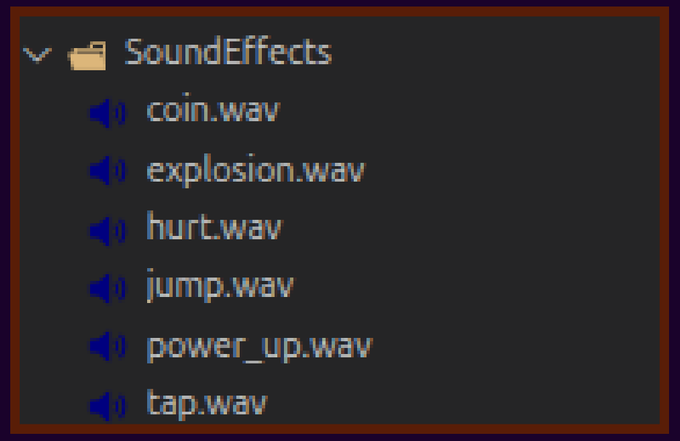
MATERIALS AND TEXTURES



The 3D objects' materials came with the imported assets, not being designed by me.

Every non-3D entity rendered in the scene is represented in the foreground using a SpriteMaterial, created through its constructor. They're essentially a walking container with which holds a 2 dimensional plane which has an image mapped to it.

SOUND




```ts
type Listener = (data?: any) => void;
export type GameEvent = "hurt" | "pickup" | "jump" | "death";

export class EventObserver {
    private listeners: Record<string, Listener[]> = {};

    constructor() {…
    }

    on(event: GameEvent | string, callback: Listener) {…
    }

    emit(event: GameEvent | string, data?: any) {…
    }

    off(event: GameEvent | string, callback: Listener) {…
    }
```

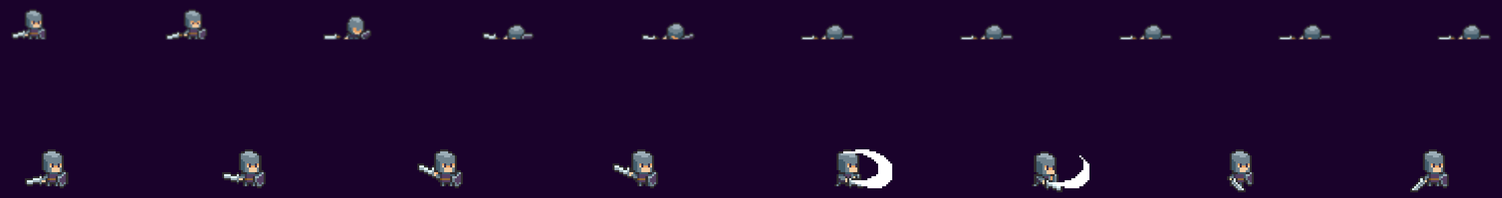
An EventObserver is responsible for handling audio events by registering system events and triggering sound effects when these events fire (e.g., when a player attacks or a slime dies).

CAMERA WORK

A single Perspective Camera is deployed onto the scene which constraints the player's view to the insides of the map model(I kept OrbitControls for debugging purposes but they're not intended to be used.

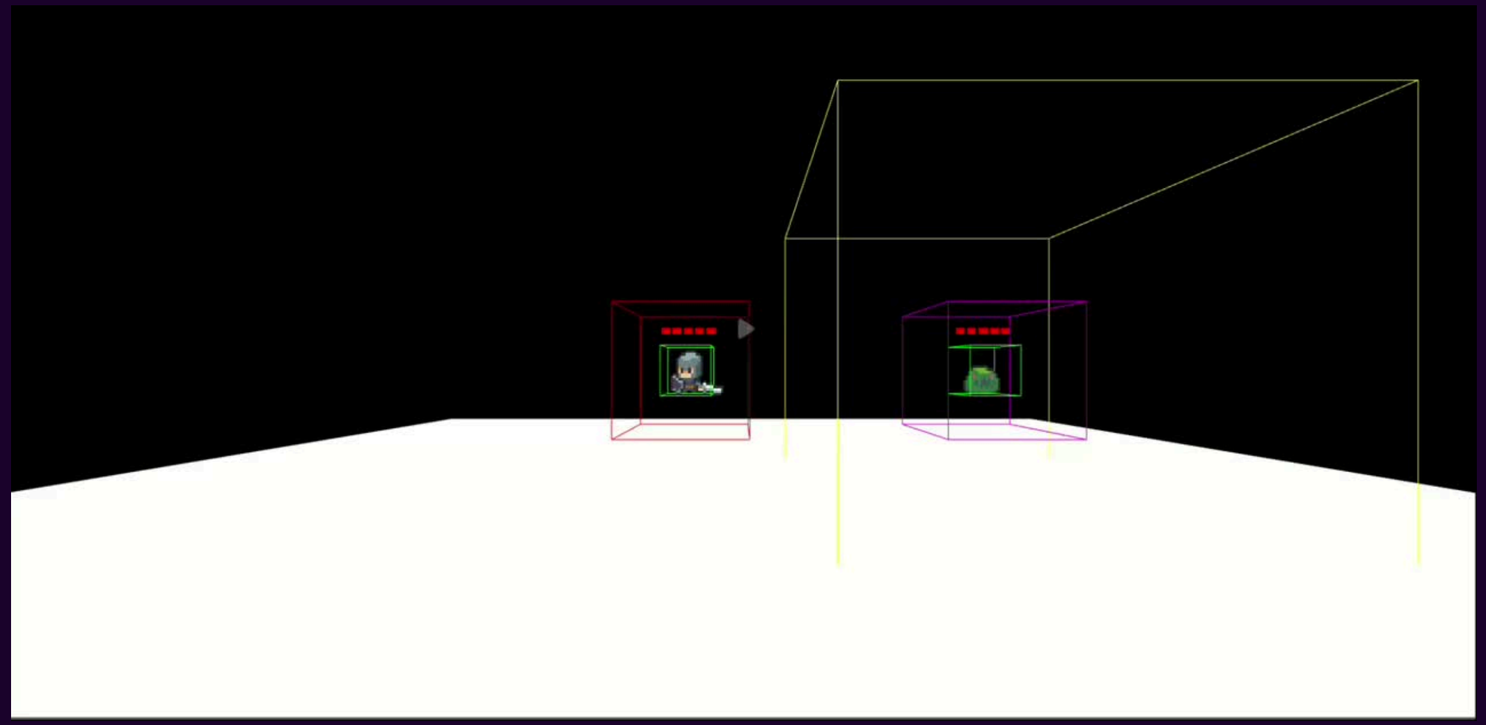
ANIMATION

Animations for 2.5D objects such as the slime, player, and health pack are implemented using sprite sheets, which contain the individual frames for each animation. A dedicated class was developed to parse these sprite sheets and cycle through the frames, creating the illusion of movement within the 3D environment.
Additionally, an AnimationLoader is responsible for loading the sprite sheets and passing them to the SpriteAnimator, which is then injected into each entity's sprite component via the SpriteAnimationManager.
No 3D animations were used; all animated elements rely exclusively on 2D sprite sheets rendered as 2.5D objects.



DEVELOPMENT

The project was developed in two main stages. The first version was implemented in JavaScript, focusing on rapidly building core gameplay and engine functionality.
However, as the project grew in complexity, the codebase became harder to maintain and extend.
This led to a full refactor into TypeScript.



DEMONSTRATION

ACKNOWLEDGEMENT OF AI USE

AI TOOLS WERE USED THROUGHOUT THE DEVELOPMENT OF THIS PROJECT TO ASSIST WITH SEVERAL TASKS. THESE INCLUDED IMPLEMENTING COLLISION DETECTION AND PHYSICS, HELPING WITH CODE REFACTORING, AND SUPPORTING THE DEPLOYMENT PROCESS USING GITHUB PAGES.
THEY WERE ALSO USEFUL FOR UNDERSTANDING AND IMPLEMENTING SPRITE SHEET ANIMATIONS WITHOUT RELYING ON A GAME ENGINE, PARTICULARLY IN MANAGING FRAME-BASED ANIMATION.
IN THE EARLY JAVASCRIPT VERSION, AI WAS USED LESS FREQUENTLY, BUT AS THE PROJECT GREW, IT HELPED ADDRESS INCREASING ARCHITECTURAL COMPLEXITY AND MAINTENANCE ISSUES.
THE FINAL ARCHITECTURE WAS MAINLY DESIGNED BY ME, WITH AI PROVIDING SUPPORT IN SPECIFIC AREAS. SOME PARTS OF THE CODE, ESPECIALLY THE MAIN FILE, COULD STILL BE FURTHER REFACTORED FOR BETTER STRUCTURE, DUE TO ITS SIZE.

PERFORMANCE & CONCLUSIONS

THE PROJECT WAS DEVELOPED TO RUN ONLY IN A WEB BROWSER, AND MOBILE DEVICES WERE NOT SPECIFICALLY CONSIDERED DURING DEVELOPMENT. AS A RESULT, COMPATIBILITY AND PERFORMANCE ON SMARTPHONES WERE NOT OPTIMISED OR TESTED.
SOME PERFORMANCE ISSUES MAY OCCUR, WITH OCCASIONAL LAG DEPENDING ON THE NUMBER OF SLIMES PRESENT IN THE SCENE, WHICH IS LIKELY DUE TO THE INCREASED COMPUTATIONAL LOAD FROM MULTIPLE ACTIVE ENTITIES AND THEIR ASSOCIATED LOGIC.

IF I WERE TO START THE PROJECT AGAIN, I WOULD HAVE A BETTER UNDERSTANDING OF WHICH DESIGN PATTERNS TO USE AND WHERE TO APPLY THEM. I WOULD ALSO CONSIDER ADDING ADDITIONAL MECHANICS, SUCH AS A BLOCKING ABILITY FOR THE SOLDIER, IMPLEMENTING VERTICALITY IN THE GAMEPLAY, AND EXPERIMENTING WITH DIFFERENT MAP DESIGNS AND ENEMY TYPES.

REFERENCES

- A COUPLE OF COURSES ON DESIGN PATTERNS AND SOFTWARE ARCHITECTURE
- SMALL INTRODUCTORY COURSE ON THREE.JS
- HTTPS://SKETCHFAB.COM/
- HTTPS://ITCH.IO/
- HTTPS://OPENGAMEART.ORG/


THANK
YOU